Hospital Management System

Project Documentation

A role-based, multi-portal hospital web application built with Django 6.1, featuring email OTP-verified authentication across all four portals.

Portals: Patient • Doctor • Receptionist • Admin

Documentation prepared: August 16, 2026

Covers: system architecture, data model, authentication & email-OTP verification, portal features, URL reference, configuration, deployment, and seed data.

Table of Contents

1. Introduction & Purpose

The Hospital Management System (HMS) is a web application that digitizes day-to-day hospital operations: patient registration and appointment booking, doctor scheduling and prescriptions, front-desk (receptionist) workflows, and system-wide administration. It is built as a single Django 6.1 project split into five apps, each owning one portal or concern, sharing one custom user model and one appointment/notification data layer.

As of this revision, every login and every new account is verified by a one-time email code sent to the user's registered address before a session is created — this applies uniformly across all four roles (patient, doctor, receptionist, admin), not just self-registered patients.

Portals covered:

- **Patient** — public site, self-registration, appointment booking, profile, notifications.
- **Doctor** — dashboard, appointment queue, prescriptions, availability/schedule management.
- **Receptionist** — front-desk patient registration, appointment booking/confirmation, doctor directory.
- **Admin** — analytics dashboard, doctor/receptionist onboarding, system-wide oversight, account status control.

2. Technology Stack

Component	Choice
`Framework`	Django 6.1
`Language`	Python 3
`Database`	SQLite locally; Postgres in production via dj-database-url + psycopg2-binary
`Static files`	WhiteNoise (CompressedManifestStaticFilesStorage)
`Config / secrets`	python-decouple, reading a gitignored .env file
`Image handling`	Pillow (profile pictures, uploads)
`WSGI server (prod)`	gunicorn
`Hosting`	Render
`Email`	Django SMTP backend via Gmail (App Password), HTML + text multipart

The stack is deliberately lean: no REST framework, no Celery/task queue, no third-party auth or forms library. All forms are raw HTML parsed directly in views via `request.POST` — the project does not use Django's `forms.py` anywhere, by convention.

3. System Architecture

The codebase is split into five Django apps under one project (`config`). `accounts` owns only the custom user model; all public pages, authentication views (login, registration, OTP verification, logout) and patient-facing features live in the `patient` app, since that app owns the shared entry point of the site.

Root URL routing (config/urls.py):

Prefix	App mounted	Notes
`admin/`	`Django built-in admin`	Stock Django admin site — separate from the custom adminpanel app
`(root)`	`patient.urls`	Public pages, auth/OTP, patient profile & appointments
`doctor/`	`doctor.urls`	Doctor portal
`receptionist/`	`receptionist.urls`	Receptionist portal
`adminpanel/`	`adminpanel.urls`	Custom staff/admin dashboard

`accounts.urls` *is empty and not included anywhere — that app contributes only the* `User` *model.*

App responsibilities at a glance:

App	Owns
`accounts`	Custom User model (email login, role field, OTP fields). No views/URLs/templates of its own.
`patient`	Public site, auth + OTP flow, patient profile, appointments, notifications.
`doctor`	Doctor dashboard, appointment queue, prescriptions, availability & schedule.
`receptionist`	Front-desk patient registration, appointment booking/confirmation, doctor directory.
`adminpanel`	Analytics dashboard, doctor/receptionist onboarding, system-wide oversight, account status.
`config`	Settings, root URL conf, WSGI application.

Each portal app (`patient`, `doctor`, `receptionist`, `adminpanel`) also defines its own `context_processors.py` exposing an unread-notification badge count, registered globally in `config/settings.py`'s template context processors.

4. Authentication, Roles & Email Verification

4.1 Custom user model & roles

`accounts.User` extends Django's `AbstractUser`, logs in by e-mail (`USERNAME_FIELD = 'email'`), and carries a single `role` field (choices: admin / doctor / receptionist / patient, default patient) that is the sole source of truth for which portal an account belongs to. There is no use of Django's Group/Permission system for portal access.

Field	Notes
`email`	EmailField(unique=True) — the login identifier

Field	Notes
`role`	admin / doctor / receptionist / patient, default 'patient'
`otp_code`	CharField(max_length=6) — current pending verification code
`otp_created_at`	DateTimeField — used with OTP_VALID_MINUTES to expire codes

4.2 Role-based access control pattern

Each portal app defines its own local decorator that wraps `@login_required` and checks `request.user.role`, redirecting home with an error message on mismatch. The same pattern is duplicated independently as `doctor_required`, `receptionist_required` and `admin_required` in their respective `views.py` files:

```
def doctor_required(view_func):
    @wraps(view_func)
    @login_required
    def wrapper(request, *args, **kwargs):
        if request.user.role != 'doctor':
            messages.error(request, 'Access restricted to doctors.')
            return redirect('home')
        return view_func(request, *args, **kwargs)
    return wrapper
```

Patient-facing views use only bare `@login_required` (no dedicated `patient_required` decorator exists) — see §10 for the implication.

4.3 Email OTP verification flow

Every login — regardless of role — is a two-step process, and every new patient self-registration routes into the same second step instead of completing immediately:

- **Step 1 (credentials):** the shared login form posts email/password/role to the `login` view. On success, a 6-digit code is generated with `secrets.randbelow`, stored on `otp_code`/`otp_created_at`, e-mailed to the user, and the target user id is stashed in `request.session['pending_otp_user_id']`. No Django session is authenticated yet.
- **Step 2 (verification):** the user is redirected to `verify_otp`, enters the code, and it is checked against the stored value with a `settings.OTP_VALID_MINUTES` (5-minute) expiry window. On success the OTP fields are cleared, `auth_login()` is called, and the user is redirected to their portal dashboard via a `ROLE_REDIRECTS` lookup (admin→admin_dashboard, doctor→doctor_home, receptionist→receptionist_dashboard; patients fall through to the home page).
- **Resend:** a `resend_otp` view regenerates and re-sends the code for the same pending session user.
- **Registration:** `register` creates the `User` (role forced to patient) and `Patient` profile, then sends an OTP and redirects into the exact same `verify_otp` step — so a new account is confirmed and signed in in one continuous flow, without a separate manual login afterwards.
- **Front-desk registration is exempt:** when a receptionist registers a walk-in patient (`receptionist.register_patient`), no OTP is sent — that account is staff-created, not self-registered, and

the patient will go through the normal OTP step the first time they log in themselves.

4.4 OTP email design

The verification code is sent via `EmailMultiAlternatives` with both a plain-text body and an HTML alternative rendered from `templates/patient/emails/otp_email.html` — a self-contained, table-based, inline-CSS layout (for email-client compatibility) using the site's actual brand blue (`#0d6efd` / `#0b5ed7`), with the code shown in a large letter-spaced monospace box and a stated expiry time.

5. Data Model

All persistent data lives in nine models across three apps, tied together by foreign keys back to the single `accounts.User` table. `adminpanel` defines no models of its own — it operates entirely on data owned by the other apps.

accounts.User — see §4.1 above for full field list.

patient.Patient

Field	Notes
`user`	OneToOne → accounts.User
`patient_id`	Auto-generated unique code, e.g. PAT000007 (assigned in an overridden save())
`registered_by`	FK → receptionist.Receptionist, nullable — set when front-desk registered
`profile_image`	ImageField, optional
`phone, date_of_birth, gender, blood_group`	Contact & demographic fields
`address, city, state, country, pincode`	Address fields
`emergency_contact_name, emergency_contact_number`	Emergency contact
`allergies, medical_history`	Free-text clinical notes
`created_at`	auto_now_add

patient.Appointment

Field	Notes
patient, doctor	FK → Patient, FK → Doctor
appointment_date, time_slot	Date + fixed time-slot string, e.g. '09:00 AM'
visit_type	new / follow-up
department	cardiology / neurology / orthopedics / pediatrics / dermatology / general
status	pending / confirmed / cancelled / completed
reason	Free-text reason for visit
Meta.unique_together	(doctor, appointment_date, time_slot) — prevents double-booking at the DB level

patient.Notification & patient.ContactMessage

Field	Notes
Notification.user	FK → accounts.User, shared model used by all four portals' notification bells
Notification.notification_type	confirmed / reminder / cancelled / general
Notification.is_read	Boolean, drives the unread badge count per portal
ContactMessage	full_name, email, phone, subject, message — public contact-form submissions

doctor.Doctor

Field	Notes
user	OneToOne → accounts.User
department, specialization, qualification	Professional profile fields
experience_years, consultation_fee	PositiveIntegerField
bio, profile_picture, license_number, registered_since	Extended profile fields
slot_duration, buffer_time, max_per_day	Scheduling configuration used when generating bookable slots

doctor.DoctorAvailability & doctor.BlockedDate

Field	Notes
`DoctorAvailability`	doctor, day (Mon–Sun), is_available, start_time/end_time — unique per (doctor, day)
`BlockedDate`	doctor, date, reason — unique per (doctor, date); dates the doctor is fully unavailable

doctor.Prescription & doctor.PrescriptionMedicine

Field	Notes
`Prescription`	doctor, patient, appointment (nullable), diagnosis, advice, follow_up_date, created_at
`PrescriptionMedicine`	prescription (FK), name, dosage, frequency (choices), duration, instructions — one row per medicine

receptionist.Receptionist

Field	Notes
`user`	OneToOne → accounts.User
`employee_id`	Auto-generated unique code, e.g. REC000003
`shift`	morning / evening / night
`phone, profile_picture, date_joined`	Contact & onboarding metadata

6. Portal Walkthrough

Each portal has its own visual identity (a distinct CSS variable namespace and accent colour: blue for patient, teal for doctor, indigo for receptionist, amber/bronze for admin) but follows an identical page layout convention: a `base.html` shell with a sidebar/topbar, a shared notification bell fed by that app's context processor, and per-page CSS/JS files under a matching static namespace.

6.1 Patient portal

- Public marketing pages (home, about, services, departments, doctor directory, contact form).
- Self-registration with immediate email OTP verification (see §4.3).
- Profile management (personal + medical info, profile photo).
- Appointment booking with real-time slot-conflict checking, own appointment history, cancellation.
- Notification centre (list, mark-all-read, dismiss).

6.2 Doctor portal

- Dashboard: today's appointments, distinct patient count, appointments missing a prescription, monthly volume, recently-seen patients.
- Appointment queue: today's list, full history, mark-complete, cancel (with patient notification).
- Patient record view scoped to patients who actually have an appointment with that doctor, with computed age.
- Prescription authoring with a dynamic multi-medicine row (name/dosage/frequency/duration/instructions per row) and a prescription history log.
- Weekly availability editor (per-weekday hours + blocked dates) and slot-duration/buffer/max-per-day scheduling config.
- Profile, edit profile, and change-password (session-preserving via update_session_auth_hash).

6.3 Receptionist portal

- Dashboard: today's appointments, patient/doctor totals, pending count, recently registered patients.
- Walk-in patient registration (no OTP — staff-initiated) and searchable patient directory/detail view.
- Appointment booking on a patient's behalf against a fixed 12-slot daily schedule, with conflict checking; confirm/cancel actions with patient notifications.
- Doctor directory showing each doctor's today-availability flag and today's appointment load.
- Notifications, profile, edit profile, change password.

6.4 Admin portal

- Analytics dashboard: total patients/doctors/receptionists/appointments, today's and pending counts, an estimated revenue figure (sum of consultation fees on completed appointments), appointment breakdowns by department and status (chart-ready, with colour mapping), and a 6-month new-patient registration trend.
- Doctor and receptionist onboarding (create account + profile) plus searchable directories and detail views.
- System-wide patient directory and a global appointment list spanning every doctor (full visibility, unlike the doctor portal's own-patients scoping).
- Account status control: activate/deactivate any non-admin account (explicitly blocked for admin targets) — this is the only account-suspension mechanism; there is no separate delete-user feature by design.
- Notifications, profile, edit profile, change password.
- This is a custom in-house dashboard, entirely separate from Django's built-in /admin/ site (which remains mounted and usable independently).

7. URL Reference

7.1 Patient app (mounted at site root)

Path	Name	Purpose
/	home	Public landing page
/about/	about	About page
/services/	services	Services listing
/departments/	department	Departments listing
/doctors/	doctors	Public doctor directory
/contact/	contact	Contact form (GET/POST)
/login/	login	Login step 1 — sends OTP
/register/	register	Patient self-registration
/logout/	logout	Logout
/forgot-password/	forgot_password	Static forgot-password page (no backend logic yet)
/verify-otp/	verify_otp	OTP verification (step 2 of login/register)
/resend-otp/	resend_otp	Resend OTP code
/patient-profile/	patient_profile	View/edit own patient profile
/book-appointment/	book_appointment	Book a new appointment
/my-appointments/	my_appointments	List own appointments
/appointments/<id>/cancel/	cancel_appointment	Cancel own appointment
/notifications/	notifications	List own notifications

7.2 Doctor app (mounted at /doctor/)

Path	Name	Purpose
	doctor_home	Dashboard
appointments/today/	today_appointments	Today's appointments
appointments/	appointment_list	Full appointment history
patients/<patient_id>/	patient_details	Patient detail/history
prescriptions/add/	add_prescription	Create prescription
prescriptions/history/	prescription_history	All prescriptions written
availability/	availability	Manage weekly schedule/blocked dates
notifications/	doctor_notifications	List notifications
profile/	doctor_profile	View / edit profile, change password

7.3 Receptionist app (mounted at /receptionist/)

Path	Name	Purpose
	receptionist_dashboard	Dashboard
patients/	receptionist_patient_list	Search/list patients
patients/register/	receptionist_register_patient	Register a walk-in patient
appointments/book/	receptionist_book_appointment	Book on behalf of a patient
appointments/<id>/confirm/	receptionist_confirm_appointment	Confirm a pending appointment
appointments/	receptionist_appointment_list	All appointments
doctors/	receptionist_doctor_list	Doctor directory with availability
profile/	receptionist_profile	View / edit profile, change password

7.4 Admin app (mounted at /adminpanel/)

Path	Name	Purpose
	admin_dashboard	Analytics dashboard
doctors/add/	admin_add_doctor	Onboard a doctor
receptionists/add/	admin_add_receptionist	Onboard a receptionist
patients/	admin_patient_list	Search/list patients
appointments/	admin_appointment_list	All appointments, system-wide
users/<id>/toggle-status/	admin_toggle_user_status	Activate/deactivate an account
profile/	admin_profile	View / edit profile, change password

8. Configuration & Environment Variables

All environment-dependent settings are read via `python-decouple`'s `config()` from a gitignored `.env` file at the project root (a documented `.env.example` ships in the repo as a template). No secret values are reproduced in this document.

Key	Purpose
SECRET_KEY	Django cryptographic signing key
DEBUG	Toggles debug mode / verbose error pages
ALLOWED_HOSTS	Comma-separated list of permitted Host headers
DATABASE_URL	If set, used by dj-database-url (e.g. Postgres on Render); falls back to local SQLite
EMAIL_BACKEND	Django email backend class — smtp in production, console for local testing
EMAIL_HOST / EMAIL_PORT / EMAIL_USE_TLS	SMTP connection details (default: smtp.gmail.com : 587, TLS)
EMAIL_HOST_USER / EMAIL_HOST_PASSWORD	Gmail address and App Password used to authenticate SMTP sends
DEFAULT_FROM_EMAIL	From-address used on outgoing OTP emails

Note: Gmail requires a 16-character App Password (not the normal account password), which in turn requires 2-Step Verification to be enabled on the sending Google account (myaccount.google.com/apppasswords). OTP_VALID_MINUTES (currently 5) is set directly in config/settings.py, not via environment variable.

9. Deployment

The project is set up to deploy to Render: gunicorn is the production WSGI server, psycopg2-binary supports a Postgres DATABASE_URL, and WhiteNoise serves compressed, cache-busted static assets without a separate web server or CDN.

Gaps worth flagging:

- No render.yaml, Procfile, or build script exists in the repository — the Render service's build/start commands must be configured manually in the Render dashboard.
- Media uploads (profile pictures, etc.) are only served by Django itself when DEBUG=True; there is no production media-serving path configured, and Render's disk is ephemeral — uploaded files would not persist across deploys without an external storage backend (e.g. S3-compatible storage).

10. Seed Data & Test Accounts

Root-level scripts populate demo data for local development (run via manage.py shell -c "exec(open('script.py').read())"); each guards against re-seeding by checking for an existing email first.

Script	Creates
seed_admin.py	One admin account

Script	Creates
`seed_doctors.py`	Nine doctor accounts + profiles across six departments
`seed_patients.py`	Ten patient accounts + profiles with varied demographics
`seed_receptionist.py`	One receptionist account
`generate_placeholder_images.py`	Programmatically generates placeholder logo/hero/avatar images with Pillow, since seeded accounts have no real uploaded photos

11. Known Limitations & Recommendations

- **Forgot password is UI-only:** the form renders but has no POST handling or password-reset email logic behind it yet.
- **Patient views lack a dedicated role decorator:** patient-facing views rely on bare `@login_required` only, unlike doctor/receptionist/admin which each check `request.user.role`. In practice this is mitigated by the UI not linking non-patients into those pages, but it is not enforced at the view layer.
- **No shared RBAC helper:** the `{role}_required` decorator logic is duplicated near-identically in three separate `views.py` files rather than factored into one shared module.
- **Media storage in production** needs an external backend before this is safe to run on Render with real user uploads (see §9).
- **No automated test suite** is present in the repository at this time.